SSH

(Secure Shell Protocol)

★ Definition

SSH is a cryptographic network protocol that provides secure remote login, command execution and file transfer between two devices over an insecure network. Client-server model, `TCP port 22`.

Secure Access
for a
Secure Network

Remember!
SSH = Secure Shell
for Secure Access

★ Purpose

- ✓ Secure remote login to servers & network devices
- ✓ Run commands on a remote system
- ✓ Transfer files securely (SCP / SFTP)
- ✓ Replace Telnet, FTP, rlogin, rsh — all plain text
- ✓ Provide `confidentiality, integrity, authentication`

★ How SSH Works?



1. Client opens a TCP connection to the server on port 22.
2. Server sends its host key; the client verifies it against `known_hosts`.
3. Both sides negotiate algorithms and perform a key exchange.
4. A shared session key is derived — `encryption starts here`.
5. The user authenticates (password or key) inside that encrypted channel.
6. Secure session established — commands and data flow encrypted.

★ Key Features

- ★ Encryption — protects data from sniffing
- ★ Authentication — proves both identities
- ★ Integrity — detects any tampering
- ★ Tunnelling — carries other protocols

★ Use Cases

- ● Remote server & network device management
- ● Secure file transfer (SCP / SFTP)
- ● CLI access to routers, switches, firewalls
- ● Automation, backups, Ansible, Git

SSH vs Telnet · Ports & Versions

★ ⚠ SSH vs Telnet — the table to memorise

Feature	SSH	Telnet
Security	Encrypted ✓	Unencrypted ✗
Default port	22 / TCP	23 / TCP
Authentication	Password, key, certificate	Password only, in clear text
Server identity	Verified by host key	Not verified — easily spoofed
File transfer	SCP, SFTP built in	None
Verdict	Use everywhere	Disable everywhere

★ SSH Ports

- Default port: 22 / TCP
- Can be changed (2222, 2200, 8022)
- If changing, use a non-privileged port (>1024)

★ SSH Versions

★ SSHv1 → obsolete and insecure

★ SSHv2 → current, secure, the only one to use

★ SSHv2 is not backward compatible with v1

Always use SSHv2 only

★ Insecure protocols SSH replaced

→ Telnet (23) — credentials in clear text

→ FTP (21/20) — credentials and data in clear

→ rlogin / rsh (513/514) — trust by IP, no crypto

⚠ Field reality

Most networks still have `transport input all` on some vty lines. Fix: `transport input ssh` on every line.

Telnet has no security at all — every login is a password broadcast.

SSH Architecture & Handshake

★ ☼ The three layers (RFC 4251)

1 · Transport Layer

Runs first over TCP 22. Version exchange, algorithm negotiation, key exchange, server authentication by host key, then encryption, integrity and optional compression. Also handles re-keying.

2 · User Authentication Layer

Runs inside the encrypted channel. Proves who the user is — password, public key, keyboard-interactive, host-based.

3 · Connection Layer

Multiplexes many channels over one connection: shell, exec, SFTP subsystem, port forwarding, X11.

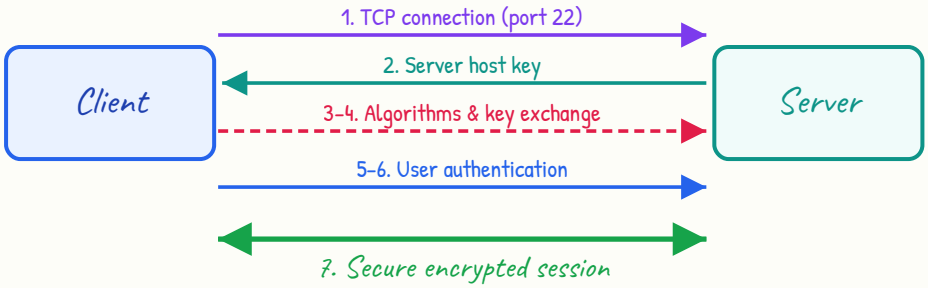
Order is fixed: Transport → Authentication → Connection. You cannot authenticate before keys exist, and you cannot open a channel before you are authenticated.

Handshake — 7 steps

1. Client connects to server (TCP 3-way handshake).
2. Server sends its host key (public).
3. Client verifies the host key against `known_hosts`.
4. Algorithms negotiated + key exchange performed.
5. Session keys derived on both sides — never transmitted.
6. User authentication takes place.
7. Encrypted session ready for data.

Exam point

The session key is derived independently at both ends, not sent across the wire. That is the whole trick of key exchange.



SSH Keys & Authentication

★ Symmetric vs Asymmetric

Symmetric key

- Same key encrypts and decrypts
- Fast — used for the session data
- Both sides derive it during key exchange
- Example: AES, ChaCha20

Asymmetric key

- Key pair: public + private
- Used for authentication, not bulk data
- Private key signs → public key verifies
- Example: Ed25519, ECDSA, RSA

★ Key generation — 6 steps

```
ssh-keygen -t ed25519 -a 100 -C "ihtiram@2026"
# RSA alternative (older devices):
ssh-keygen -t rsa -b 4096
```

1. Run `ssh-keygen` with the key type
2. Choose the file location — default `~/.ssh/`
3. Enter a passphrase (strongly recommended)
4. Private key stays secret, never leaves the machine
5. Public key `.pub` is the one you share
6. Install it: `ssh-copy-id user@host`

Authentication methods

- Password — simple, phishable, brute-forceable
- Public key — recommended default
- Keyboard-interactive — the hook for OTP / MFA
- Host-based — trusts the client machine

Tip

Prefer Ed25519 over RSA — smaller, faster, safer defaults. Use RSA 4096 only when the far-end device is old.

★ Public-key authentication flow

1. Client generates the key pair (private + public).
2. Public key is copied to the server → stored in `~/.ssh/authorized_keys`.
3. Client connects and signs a challenge with its private key.
4. Server verifies that signature using the stored public key.
5. If valid → access granted, no password sent.

File permissions

Path	Mode
~/.ssh	700
~/.ssh/id_*	600
~/.ssh/authorized_keys	600
~/.ssh/*.pub	644

Wrong permissions = keys silently refused, with no useful message on the client. Check `/var/log/auth.log`.

Host Keys & known_hosts

★ Why the client checks the server

Before you type any password, the client authenticates the server using its host key. On the first connection you have never seen that key, so SSH shows you the fingerprint and asks. If you type "yes" without comparing it to anything, you have verified nothing.

```
The authenticity of host 'R1 (10.10.10.1)' can't be established.
ED25519 key fingerprint is SHA256:9qXk3nB2v…8dQe.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
```

★ Doing it properly

→ Read the real fingerprint on the device console first:

```
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
```

→ Compare it with what the client shows, then accept

→ Pre-load `known_hosts` on jump hosts for the whole estate

★ ⚠ "Host key verification failed"

```
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
Offending key in /home/mickey/.ssh/known_hosts:14
```

Innocent causes: device replaced, OS reinstalled, IP reused by another box, config rebuilt.

Guilty cause: someone is sitting between you and the device.

Correct procedure

1. Verify the new fingerprint from the console or change record.
2. Then remove the old entry: `ssh-keygen -R 10.10.10.1`
3. Reconnect and compare again. Never just delete the line and carry on.

Useful commands

```
ssh-keygen -lf ~/.ssh/known_hosts
ssh-keygen -F 10.10.10.1
ssh-keygen -R 10.10.10.1
ssh-keyscan -t ed25519 10.10.10.1
ssh -o VisualHostKey=yes host
```

Key files to know

Path	What it is
/etc/ssh/sshd_config	Server config
/etc/ssh/ssh_config	Client config (system)
~/.ssh/config	Client config (user)
~/.ssh/id_ed25519	Private key
~/.ssh/id_ed25519.pub	Public key
~/.ssh/authorized_keys	Keys allowed in (server)
~/.ssh/known_hosts	Host keys you trust

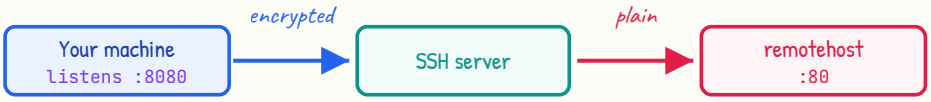
⚠ Never in production

`StrictHostKeyChecking=no` with `UserKnownHostsFile=/dev/null`. It is pasted into pipelines everywhere and it disables the only protection against a man-in-the-middle.

SSH Tunnelling + Useful Commands

★ 1 · Local Port Forwarding (-L)

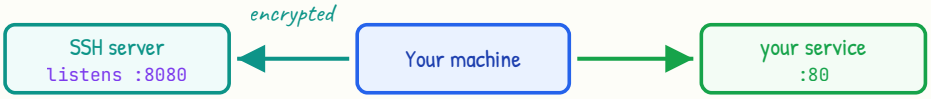
```
ssh -L 8080:remotehost:80 user@ssh_server   # browse http://localhost:8080
```



Reach a service you cannot route to — the SSH server makes the final hop.

★ 2 · Remote Port Forwarding (-R)

```
ssh -R 8080:localhost:80 user@ssh_server   # on the server: curl http://localhost:8080
```



Expose something on your side to the remote network.

★ 3 · Dynamic Port Forwarding (-D)

```
ssh -D 1080 user@ssh_server   # SOCKS5 proxy — destination chosen per connection
```

The rule that ends the confusion

The left side of the argument is where the socket opens. The right side is resolved from the opposite end. That is why `localhost` means a different machine in `-L` and `-R`.

★ Useful SSH commands

`ssh user@ip`	Connect to a host	`scp file user@ip:/path/`	Copy a file to remote
`ssh -p 2222 user@ip`	Non-default port	`scp user@ip:/path/file .`	Copy from remote
`ssh -i key user@ip`	Use a private key	`sftp user@ip`	Interactive file transfer
`ssh -J bastion user@ip`	Jump through a bastion	`rsync -avz -e ssh dir/ ip:/dst/`	Sync directories
`ssh -v user@ip`	Verbose / debug	`ssh-copy-id user@ip`	Install your public key

Cisco IOS / IOS-XE SSH Config

★ Configuration — order matters

```
! 1. hostname + domain (the key is named host.domain)
hostname R1
ip domain-name corp.local

! 2. local user with full privilege
username ihtiram privilege 15 secret StrongPass123

! 3. generate the key pair (2048 min, 4096 better)
crypto key generate rsa modulus 4096

! 4. SSH parameters
ip ssh version 2
ip ssh time-out 60
ip ssh authentication-retries 3

! 5. stronger algorithms (IOS-XE 16.x+)
ip ssh server algorithm mac hmac-sha2-256 hmac-sha2-512
ip ssh server algorithm encryption aes256-ctr

! 6. restrict who may reach the vty
ip access-list standard MGMT-IN
 permit 10.20.0.0 0.0.0.255
 deny   any log

! 7. the vty lines — the actual door
line vty 0 15
 transport input ssh
 access-class MGMT-IN in
 login local
 exec-timeout 10 0
```

Verify

```
show ip ssh
show ssh
show crypto key mypubkey rsa
show users
show run | section line vty
debug ip ssh detail
```

`show ip ssh` must report `SSH Enabled - version 2.0`. If it says Disabled, the key generation step did not complete.

IOS as an SSH client

```
R1# ssh -l admin 10.10.10.2
R1# ssh -l admin -p 2222 10.10.10.2
```

⚠ Gotchas

→ Changing hostname or domain-name invalidates the RSA key — SSH stops. Regenerate.

→ `transport input all` on any vty leaves Telnet alive — set all 16 lines.

→ No `login local` and no AAA = keys exist but you still cannot log in.

→ Old IOS may need modern clients to allow legacy algorithms — treat that as a reason to upgrade, not a permanent config.

Before changing vty or SSH config on a remote device — open a second session first.

Huawei VRP SSH (STelnet)

★ Configuration

```
# 1. enter system view
system-view
sysname S6730-CORE

# 2. generate the local key pair
rsa local-key-pair create        # choose 2048

# 3. AAA — create the local user
aaa
 local-user ihtiram password irreversible-cipher Huawei@123
 local-user ihtiram service-type ssh
 local-user ihtiram privilege level 15
 quit

# 4. create the SSH user and its service
ssh user ihtiram
ssh user ihtiram authentication-type password
ssh user ihtiram service-type stelnet    # or sftp / all

# 5. enable the servers
stelnet server enable
sftp server enable

# 6. VTY must permit SSH only
user-interface vty 0 4
 authentication-mode aaa
 protocol inbound ssh
 user privilege level 15
 quit

# 7. hardening
ssh server timeout 60
ssh server authentication-retries 3
ssh server compatible-ssh1x disable
save
```

Verify

```
display ssh server status
display ssh user-information
display rsa local-key-pair public
display users
display user-interface vty 0 4
```

VRP as a client

```
stelnet 10.20.0.1
sftp 10.20.0.9
```

⚠ Field notes

→ The user needs both `service-type ssh` under AAA and an `ssh user … service-type` line. Missing one = "authentication failed" with a correct password.

→ `protocol inbound all` is the default on many images — it leaves Telnet open.

→ On USG firewalls SSH must also be allowed on the interface: `service-manage ssh permit`.

→ Regenerating the key pair drops all live SSH sessions — schedule it.

Huawei calls the SSH server STelnet. Same protocol, different name.

Juniper Junos + Linux Server

★ Juniper Junos

```
configure
set system services ssh
set system services ssh protocol-version v2
set system services ssh root-login deny
set system services ssh connection-limit 10
set system services ssh rate-limit 4
set system services ssh ciphers aes256-ctr
set system services ssh macs hmac-sha2-256
set system login user ihtiram class super-user
set system login user ihtiram authentication \
  ssh-ed25519 "ssh-ed25519 AAAAC3Nz… mickey"

commit confirmed 5   # auto rollback in 5 min
# test from a second terminal, then:
commit and-quit
```

```
show configuration system services ssh
show system users
show system connections | match 22
```

Why Junos is safer

`commit confirmed` rolls the change back by itself if you lose access. IOS has no equivalent — that is why the second-session habit matters more there.

★ Linux — sshd_config

```
Port 22
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
AllowGroups sshusers netops
MaxAuthTries 3
LoginGraceTime 30
ClientAliveInterval 300
ClientAliveCountMax 2
X11Forwarding no
AllowTcpForwarding no
LogLevel VERBOSE
Banner /etc/issue.net
Ciphers aes256-ctr,aes256-gcm@openssh
MACs hmac-sha2-512,hmac-sha2-256
```

Apply it safely

```
sudo sshd -t           # 1 syntax check
sudo sshd -T           # 2 effective config
sudo systemctl reload ssh  # 3 keep 1st session
ssh -v user@host     # 4 test in 2nd term
```

Permissions

`~/.ssh` = 700 · `authorized_keys` = 600

Home directory must not be group-writable.

Windows + Advanced Features

★ Windows OpenSSH

```
# PowerShell as Administrator
Add-WindowsCapability -Online `
  -Name OpenSSH.Server~~~~0.0.1.0
Start-Service sshd
Set-Service -Name sshd -StartupType Automatic
Get-Service sshd
```

- Config: `C:\ProgramData\ssh\sshd_config`
- Same hardening options as Linux
- Firewall rule needed for inbound TCP 22

⚠ Windows trap

For members of the Administrators group, keys in `%USERPROFILE%\.ssh\authorized_keys` are ignored. Use `C:\ProgramData\ssh\administrators_authorized_keys` instead, or you will chase a working config that never authenticates.

★ Advanced features

★ ProxyJump — `ssh -J bastion target`. Your key stays on your laptop; the bastion only relays bytes.

★ Agent forwarding — `ssh -A`. Avoid it: root on the middle host can use your key as you.

★ Multiplexing — `ControlMaster` reuses one connection; big speed-up for Ansible.

★ SSH certificates — one CA signs every key, access expires by itself. Replaces copying keys to hundreds of devices.

★ fail2ban — bans IPs after repeated failed logins.

```
# ~/.ssh/config — type less, fail less
Host bastion
  HostName bastion.corp.sa
  User ihtiram
  IdentityFile ~/.ssh/id_ed25519

Host 10.10.*
  ProxyJump bastion
  User admin
  IdentitiesOnly yes
```

`ssh -G host` shows the settings that will actually be used — without connecting.

ProxyJump instead of agent forwarding. Keys instead of passwords. Both are one line of config.

Troubleshooting SSH

★ Common errors, causes & fixes

Error	Cause	Fix
Connection refused	Host reached, nothing listening on 22	Start the SSH service, check the port
Connection timed out	No path — ACL, firewall, wrong IP/VRF	`ping`, `nc -zv host 22`, check ACL / security policy
Permission denied (publickey)	Wrong key, key not installed, bad permissions, wrong username	`ssh -v`, fix `700/600`, check `authorized_keys`
Too many authentication failures	Agent offered more keys than MaxAuthTries	`IdentitiesOnly yes` + explicit `IdentityFile`
Host key verification failed	Host key changed or IP reused	Verify out of band, then `ssh-keygen -R <host>`
No matching cipher / kex / host key	Modern client vs legacy device	Allow the legacy algorithm temporarily, then upgrade the device
Broken pipe after idle	NAT or firewall idle timeout	`ServerAliveInterval 30` on the client

★ Work the layers in order

1. L3 — `ping`, route, correct VRF?
2. L4 — `nc -zv host 22`. Refused = daemon; timeout = filter.
3. Banner — `nc host 22`. Seeing `SSH-2.0-…` means TCP and the daemon are fine.
4. Crypto — `ssh -vv`, look for algorithm mismatch.
5. Host key — known_hosts conflict?
6. Identity — user, key, permissions, AllowUsers.

Verbosity & server logs

```
ssh -v    # auth decisions
ssh -vv   # + algorithms
ssh -vvv  # + packet level
```

```
journalctl -u ssh -f
tail -f /var/log/auth.log   # Debian
tail -f /var/log/secure     # RHEL
```

One line splits every SSH problem:

`Authentications that can continue…`

fails before it → transport · fails after it → identity

Best Practices + Final Notes

★ SSH best practices

- ✓ Key-based authentication only
- ✓ Disable root / Administrator login
- ✓ Restrict source addresses with ACL or firewall
- ✓ Strong algorithms only, legacy removed
- ✓ Use passphrases on private keys
- ✓ Log to a central server and review it

Honest note

Changing port 22 to 2222 cuts log noise — it stops no targeted attacker.

★ Modern algorithms

Type	Use
KEX	curve25519-sha256, ecdh-sha2-nistp384
Ciphers	aes256-gcm, chacha20-poly1305, aes256-ctr
MACs	hmac-sha2-512-etm, hmac-sha2-256
Host key	ssh-ed25519, rsa-sha2-512

⚠ Retire on sight

`3des-cbc` · any `*-cbc` · `hmac-md5` · `hmac-sha1` · `diffie-hellman-group1-sha1` · `ssh-dss` (DSA) · SSHv1

★ Memory hooks

-L = Local listener, look out from me · -R = Remote listener, reach back to me · -D = Decide later (dynamic)
Layers: Transport → Authentication → Connection · Port 22 · `~/.ssh` = 700 · private key = 600

SSH = Secure access · Strong authentication · Safe data transfer

Use it wisely. Stay secure.

— Ihtiram Ullah · @gulf_networking · Riyadh, KSA